

Employee Management System

Node.js Course-End Project Write-Up

1. Project Overview

The Employee Management System is a Node.js and Express application created to manage employee records through both a browser interface and REST-style API endpoints.

The application allows an administrator to:

- Sign up and log in
- View an employee directory
- Add employee records
- Update employee information
- Delete employee records
- Test CRUD operations through Postman
- Run end-to-end browser tests with Cypress

Employee and login data are stored in JSON files rather than a database, as required by the project instructions.

2. Project Requirements

The project was developed to satisfy the following course requirements:

1. Initialize a Node.js project with npm
2. Use the Node.js fs module to read and write employee data
3. Use Express.js, EJS, and cookie-parser
4. Implement CRUD API routes
5. Create and visually run Cypress tests

The completed application meets each of these requirements.

3. Technologies Used

- Node.js
- Express.js
- EJS
- JavaScript
- HTML and CSS
- Node.js fs module

- cookie-parser
 - JSON
 - Cypress
 - Postman
 - npm
 - Git and GitHub
-

4. Application Design

The project provides two ways to manage employees.

Browser Interface

The browser interface uses EJS templates rendered by Express.

The main pages include:

- Login
- Sign up
- Employee directory
- Add employee
- Manage employees

The Manage Employees page includes both update and delete controls so administrators can manage records from one view.

REST API

The application also includes API routes that return JSON and can be tested with Postman.

Method	Route	Purpose
GET	/api/employees	Retrieve all employees
GET	/api/employees/:employeeId	Retrieve one employee
POST	/api/employees	Create an employee
PATCH	/api/employees/:employeeId	Update selected fields
DELETE	/api/employees/:employeeId	Delete an employee

The browser routes return rendered HTML, while the API routes return JSON responses and HTTP status codes.

5. Data Storage

The application does not use a database.

Data is stored in:

employees.json

logins.json

The Node.js fs module reads and writes these files.

Each employee record includes:

```
{  
  "employeeId": 1,  
  "name": "Maya Thompson",  
  "emailId": "maya.thompson@company.com",  
  "position": "Front-End Developer",  
  "department": "Engineering",  
  "contact": "314-555-0101",  
  "location": "St. Louis, MO",  
  "joinDate": "2025-08-12"  
}
```

Employee IDs are generated by the server based on the highest existing ID.

This avoids relying on an employee's position in the JSON array, which can change when another record is deleted.

6. Shared CRUD Logic

The browser forms and API routes perform many of the same operations. To avoid duplicating logic, shared functions were created for employee management.

```
createEmployee(employeeData);
```

```
updateEmployee(employeeId, employeeData);
```

```
deleteEmployee(employeeId);
```

Additional helper functions include:

```
readEmployees();
```

```
writeEmployees();
```

```
getNextEmployeeId();
```

```
findEmployeeIndexById();
```

This design allows the same validation and file-writing logic to be used by both interfaces.

EJS routes → HTML responses

API routes → JSON responses

This refactoring was one of the most important improvements made during development.

7. Validation and Error Handling

The application includes server-side validation for employee operations.

Duplicate Employee Emails

Employee email addresses must be unique.

When a duplicate email is submitted, the application returns:

400 Bad Request

The EJS interface displays an error message in the page, while the API returns a JSON response.

```
{  
  "message": "Employee already exists"  
}
```

Missing Employees

If an employee ID does not match an existing record, the application returns:

404 Not Found

```
{  
  "message": "Employee not found"  
}
```

Successful Requests

- Employee creation returns 201 Created
- Successful retrieval, updates, and deletions return 200 OK

The update route uses PATCH, allowing Postman to send only the fields that need to change.

8. Cookie Usage

The application uses cookie-parser to read the email cookie created after a successful login.

```
res.cookie("emailId", emailId, {  
  httpOnly: true,  
  sameSite: "lax"  
});
```

The cookie is read through:

req.cookies.emailId

The email is then placed in res.locals so it can be displayed in the EJS pages.

```
res.locals.emailId = req.cookies.emailId || "";
```

For this educational project, the cookie identifies the current displayed user. It is not intended to be a complete production authentication system.

9. Testing

Postman

All CRUD API routes were tested through Postman.

Testing included:

- Retrieving all employees
- Retrieving one employee by ID
- Creating a new employee
- Partially updating an employee
- Deleting an employee
- Confirming HTTP response statuses
- Confirming duplicate-email validation
- Confirming missing-employee errors

Cypress

Cypress was configured for end-to-end browser testing.

The current tests verify that:

- The login page displays correctly
- Invalid credentials show an error
- Valid credentials allow login
- A new employee can be added

The tests were run in the Cypress visual browser environment and completed successfully.

10. Development Challenges and Learning Outcomes

Several concepts became clearer while building this project.

Separating Browser Routes and API Routes

The project helped clarify that the same business operation can be accessed through different interfaces.

For example, employee creation can happen through:

POST /addEmployeeInFile

or:

POST /api/employees

The first returns an EJS page. The second returns JSON.

Both routes now use the same shared creation function.

Stable Employee IDs

The initial update and delete logic relied on array indexes.

This was changed so records are located through a stable employeeid. Deleting one employee no longer changes how other employees are identified.

Cypress Testing

I learned how Cypress runs separately from the Express application and interacts with the browser interface through a dedicated test file.

11. Use of Generative AI

Generative AI was used as a development-support and learning tool during this project.

It assisted with:

- Explaining Express, EJS, cookies, and REST API concepts
- Reviewing route structure
- Identifying repeated CRUD logic
- Refactoring employee operations into shared functions
- Troubleshooting Postman request methods
- Setting up Cypress tests
- Organizing CSS
- Reviewing project documentation

All code was implemented, reviewed, tested, and adjusted within the project environment.

12. Known Limitations

This application was created for educational purposes and is not production-ready.

Current limitations include:

- JSON files are used instead of a database
- File operations are synchronous

- Passwords are stored as plain text
- Authentication is limited
- Routes are not protected through sessions or authorization middleware
- Cypress creation tests add records to the local JSON file

A production version would use:

- A database such as MongoDB or MySQL
- Password hashing
- Secure sessions or token authentication
- Protected routes
- Environment variables
- Formal request-validation middleware
- Asynchronous data access

13. Conclusion

The completed Employee Management System satisfies the required Node.js course-end project tasks.

The project demonstrates:

- npm project setup
- Express routing
- EJS rendering
- JSON file persistence
- Cookie parsing
- CRUD API development
- Validation and HTTP status handling
- Postman testing
- Cypress end-to-end testing
- Shared business logic
- GitHub source control

The most valuable learning outcome was understanding how browser-based routes and API routes can share the same core logic while returning different response formats.